

Real-Time Image-Based Lighting with the Split-Sum Approximationnnnn

Vid Potočnik¹

¹University of Ljubljana, Faculty of Computer and Information Science, Slovenia

Abstract

We present a real-time rendering pipeline that uses image-based lighting (IBL) to evaluate the contribution of an HDR environment map to physically based shading. The pipeline precomputes a diffuse irradiance cubemap by cosine-weighted hemisphere convolution, a specular prefiltered environment cubemap with mip-level encoded roughness via GGX importance sampling, and a two-dimensional BRDF integration lookup table using the split-sum approximation of Karis. At render time the diffuse and specular IBL terms are evaluated with three texture fetches and a Schlick–Fresnel modulation. We implement the pipeline in OpenGL 3.3 and analyse the trade-offs between preprocessing time, memory cost, and rendering accuracy across three HDR environments.

CCS Concepts

•Computing methodologies → Rendering; Reflectance modeling;

1. Introduction

Realistic surface appearance under arbitrary illumination depends strongly on the surrounding environment. Image-based lighting (IBL), introduced by Debevec [Deb98] and later codified as a practical technique [Deb06], uses a captured high-dynamic-range (HDR) panoramic image as the sole light source, evaluating the rendering equation against the panoramic radiance distribution directly. A naive evaluation requires Monte Carlo integration per pixel, which is incompatible with real-time constraints.

Karis [Kar13] proposed a *split-sum* approximation that decomposes the specular IBL integral into a precomputed prefiltered environment map and a 2D scale-bias lookup table. Combined with the standard cosine-weighted irradiance map for the diffuse term [RH01], this reduces IBL evaluation to three constant-time texture fetches per pixel and is the foundation of the real-time physically based rendering (PBR) pipelines used in modern game engines [Bur12].

This work implements and analyses such a pipeline for the GGX microfacet BRDF [WMLT07]. Section 2 reviews the underlying theory. Section 3 describes the four preprocessing passes and the per-frame shading. Section 4 describes the implementation. Section 5 reports preprocessing times, rendering performance, and a qualitative discussion of the accuracy/performance trade-offs.

2. Background

2.1. The rendering equation under environmental lighting

The outgoing radiance L_o at a surface point in direction ω_o is

$$L_o(\omega_o) = \int_{\Omega} f_r(\omega_i, \omega_o) L_i(\omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i, \quad (1)$$

where L_i is the incoming radiance, f_r is the bidirectional reflectance distribution function (BRDF), $\mathbf{n}$ is the surface normal, and Ω is the upper hemisphere. Under image-based lighting the scene is enclosed by an HDR panoramic image at infinity; $L_i(\omega_i)$ is read directly from this image, typically resampled into a cubemap to allow constant-time GPU access.

2.2. Cook–Torrance microfacet BRDF

We adopt the Cook–Torrance microfacet BRDF [CT82] with the GGX normal distribution function [WMLT07], the height-correlated Smith geometry term [Hei14], and the Schlick approximation of the Fresnel term:

$$f_r = \underbrace{\frac{(1-F)(1-m)\mathbf{c}}{\pi}}_{\text{Lambertian}} + \underbrace{\frac{D(\mathbf{h})F(\omega_o, \mathbf{h})G(\omega_i, \omega_o)}{4(\mathbf{n} \cdot \omega_i)(\mathbf{n} \cdot \omega_o)}}_{\text{specular}}. \quad (2)$$

Here $\mathbf{h}$ is the half-vector, $m \in [0, 1]$ is the metallic factor, $\mathbf{c}$ the base colour (albedo), and F_0 the reflectance at normal incidence: $F_0 = \text{mix}(0.04, \mathbf{c}, m)$. The factor $(1-F)$ enforces energy conservation between the diffuse and specular lobes, and the factor $(1-m)$ removes the diffuse contribution for metallic surfaces.

2.3. The split-sum approximation

Equation (1) cannot be evaluated analytically against an HDR environment, but it admits a useful factorisation for the two lobes of (2).

Diffuse term. For a Lambertian surface, f_r is constant and the integral reduces to the cosine-weighted average of incoming radiance,

$$E(\mathbf{n}) = \int_{\Omega} L_i(\omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i. \quad (3)$$

E depends only on the surface normal and can be precomputed once into a cubemap indexed by $\mathbf{n}$, the *irradiance map*. The angular bandlimit imposed by the cosine kernel is low, so a 32×32 resolution per face is sufficient.

Specular term. The specular integrand does not separate cleanly, because f_r^{spec} depends jointly on ω_i and ω_o . Karis [Kar13] approximates it as a product of two simpler integrals:

$$\int_{\Omega} f_r^{\text{spec}}(\omega_i, \omega_o) L_i(\omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i \approx \underbrace{\left(\frac{\sum_{k=1}^N L_i(\omega_k) (\mathbf{n} \cdot \omega_k)}{\sum_{k=1}^N (\mathbf{n} \cdot \omega_k)} \right)}_{\text{(I) prefiltered environment}} \cdot \underbrace{\int_{\Omega} f_r^{\text{spec}}(\omega_i, \omega_o) (\mathbf{n} \cdot \omega_i) d\omega_i}_{\text{(II) BRDF integral}}. \quad (4)$$

Factor (I) depends on the reflection direction $\mathbf{r} = \text{reflect}(-\omega_o, \mathbf{n})$ and roughness α . It is precomputed into a roughness-indexed sequence of cubemaps, encoded as mip levels of a single cubemap texture: mip 0 holds the mirror-reflection case ($\alpha = 0$) and mip 4 holds the fully rough case ($\alpha = 1$). The samples ω_k are drawn by importance-sampling the GGX NDF using a Hammersley quasi-random sequence.

Factor (II) depends only on $(\mathbf{n} \cdot \omega_o, \alpha)$ and admits a further algebraic simplification by isolating the Fresnel term. Substituting $F(\omega_o, \mathbf{h}) = F_0 + (1 - F_0)(1 - \omega_o \cdot \mathbf{h})^5$ yields

$$\int_{\Omega} f_r^{\text{spec}}(\mathbf{n} \cdot \omega_i) d\omega_i = F_0 A(\mathbf{n} \cdot \omega_o, \alpha) + B(\mathbf{n} \cdot \omega_o, \alpha), \quad (5)$$

where A and B are scalar functions of two arguments. They are precomputed into a single 2D lookup table, the *BRDF LUT*, with the two channels of an RG16F texture storing A and B respectively.

At render time the specular IBL evaluation collapses to two texture fetches and a multiply-add, independent of the environment complexity.

3. Pipeline

The pipeline (Fig. 1) consists of four preprocessing passes executed once when an HDR environment is loaded, and a single fragment shading pass per frame.

3.1. Equirectangular-to-cubemap conversion

The HDR file is loaded as a 2D texture in equirectangular projection (longitude $\times$ latitude). A unit cube is rendered six times into the faces of a 512^2 cubemap; the fragment shader samples the

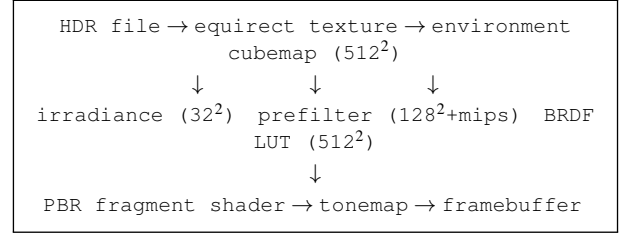


Figure 1: Pipeline overview. The HDR equirectangular image is converted into a cubemap, from which three precomputed structures are derived; the rendering pass combines them with a Cook–Torrance Fresnel response.

equirectangular texture using the texel direction $\mathbf{d} = \text{localPos}$ converted to spherical UVs:

$$u = \frac{1}{2\pi} \arctan(\mathbf{d}_z, \mathbf{d}_x) + 0.5, \quad v = \frac{1}{\pi} \arcsin(\mathbf{d}_y) + 0.5. \quad (6)$$

Storage is GL_RGB16F to retain the HDR range across the full pipeline.

3.2. Diffuse irradiance convolution

For each output texel of a 32^2 cubemap face, the upper hemisphere around the texel’s direction is integrated by a uniform Riemann sum on (ϕ, θ) with a step of 0.025 rad ($\approx 1,300$ samples per texel). The integrand is $L_i(\omega) \sin \theta \cos \theta$; the multiplicative π from the diffuse normalisation is applied at the end. The low output resolution suffices because the cosine kernel acts as a low-pass filter that removes high-frequency content.

3.3. GGX prefiltered environment map

The prefiltered cubemap is allocated at 128^2 with five mip levels and $\text{GL_LINEAR_MIPMAP_LINEAR}$ filtering. For each mip level $i \in \{0, \dots, 4\}$ the roughness is set to $\alpha = i/4$. The fragment shader, given the output direction $\mathbf{n}$ (assumed equal to $\mathbf{v}$ and $\mathbf{r}$ following Karis), draws N samples (default $N = 1024$) from a Hammersley sequence [HH64], transforms them into half-vectors $\mathbf{h}$ through the GGX inverse-CDF, computes $\mathbf{L} = \text{reflect}(-\mathbf{v}, \mathbf{h})$, and accumulates the environment radiance weighted by $(\mathbf{n} \cdot \mathbf{L})$.

Listing 1: GGX importance sampling for the prefilter pass.

```

vec3 importanceSampleGGX(vec2 Xi, vec3 N, float roughness) {
    float a = roughness * roughness;
    float phi = 2.0 * PI * Xi.x;
    float ct = sqrt((1.0 - Xi.y) / (1.0 + (a*a - 1.0)*Xi.y));
    float st = sqrt(1.0 - ct*ct);
    vec3 H = vec3(cos(phi)*st, sin(phi)*st, ct);
    // ...transform from tangent space into world space using N...
    return normalize(tangent*H.x + bitangent*H.y + N*H.z);
}
  
```

3.4. BRDF lookup table

A 512×512 RG16F texture is rendered once at startup with a single full-screen quad. The fragment shader evaluates equation (5) by importance-sampling the GGX NDF with 1024 Hammersley samples and accumulating $(1 - F_c) G_{\text{vis}}$ and $F_c G_{\text{vis}}$ separately into

[screenshot of the ImGui panel]

Figure 2: *ImGui control panel. The user adjusts material parameters, exposure, the tonemapping operator, scene composition (single object or material grid), the active HDR environment, and the four IBL preprocessing resolutions.*

the two output channels, where $F_c = (1 - \mathbf{v} \cdot \mathbf{h})^5$ and G_{vis} is the visibility-weighted Smith geometry term. The result is material-independent and need not be regenerated when the environment or the material changes.

3.5. Rendering pass

Per pixel the fragment shader (Listing 2) computes the diffuse contribution with one cubemap fetch, the specular contribution with two (the prefiltered map sampled at $\text{LOD} = \alpha \cdot 4$ and the BRDF LUT at $(\mathbf{n} \cdot \mathbf{v}, \alpha)$), and combines them with a roughness-modulated Schlick Fresnel response [Laz13] that avoids edge darkening at high roughness.

Listing 2: *Core of the per-pixel IBL evaluation.*

```
vec3 F = fresnelSchlickRoughness(NdotV, F0, roughness);
vec3 kD = (1.0 - F) * (1.0 - metallic);

vec3 irradiance = texture(irradianceMap, N).rgb;
vec3 diffuse = kD * irradiance * albedo;

vec3 prefiltered = textureLod(prefilterMap, R, roughness * 4.0).
    rgb;
vec2 brdf = texture(brdfLUT, vec2(NdotV, roughness)).rg;
vec3 specular = prefiltered * (F * brdf.x + brdf.y);

color = (diffuse + specular) * exposure;
color = tonemap(color);
```

The final colour passes through a tonemapping operator (Reinhard, ACES filmic, or Uncharted 2 [Hab10]) and gamma correction before being written to the framebuffer.

4. Implementation

The application is implemented in C++17 against OpenGL 3.3 core profile, with GLFW for window/context management, GLAD as the function loader, GLM for linear algebra, stb_image for HDR decoding, and Dear ImGui for the parameter UI. All map dimensions and the prefilter sample count are exposed through ImGui (Fig. 2); modifying any of them triggers regeneration of the affected map without restarting the application. Three primitive meshes (sphere, cube, torus) and a 7×7 roughness-metallic grid mode are provided for visual inspection. The source tree is approximately 1.5 kLOC of C++ and 350 LOC of GLSL, and builds to a single executable through CMake.

5. Results

5.1. Qualitative evaluation

Figure 3 shows the 7×7 material grid under *abandoned_parking* (rows: metallic $0 \rightarrow 1$; columns: roughness $0.05 \rightarrow 1$). Dielectric materials (top row) exhibit the expected

[7×7 material grid under *abandoned_parking*]

Figure 3: *Material grid. Rows top-to-bottom: increasing metallic. Columns left-to-right: increasing roughness. A single HDR environment (*abandoned_parking*) is used for all 49 spheres.*

[mirror sphere under three HDR environments]

Figure 4: *Mirror sphere ($\alpha = 0.05$, $m = 1$, white albedo) under three HDR environments: *abandoned_parking*, *studio_small_03*, and *kloppenheim_06* (left to right).*

Fresnel rim brightening and roughness-dependent reflection blur. Pure metals (bottom row) show a coloured specular response controlled by the albedo, with no diffuse contribution. Intermediate rows correctly interpolate between the two regimes following the metallic workflow.

Figure 4 contrasts a single mirror sphere under three HDR environments. The pipeline correctly reproduces the overcast soft lighting of *abandoned_parking*, the studio key-and-fill lighting of *studio_small_03*, and the strong sun-and-sky contrast of *kloppenheim_06*. Because L_i is stored at GL_RGB16F , the sun disk in *kloppenheim_06* retains its high dynamic range and produces a clipped highlight after tonemapping, matching the behaviour of physically captured reflections.

5.2. Preprocessing performance

Wall-clock preprocessing times are measured around a `glFinish()` barrier to capture the actual GPU execution cost. Measurements are taken on an Apple Silicon Mac with the OpenGL 4.1 Metal driver (Metal driver version 90.5). Table 1 sweeps the four independent parameters of the pipeline to isolate the cost of each pass.

Four observations follow directly from the sweep. First, doubling the prefilter output resolution from 128^2 to 256^2 (rows 3 $\rightarrow$ 4) increases the prefilter pass from 21.8 to 33.5 ms ($1.5\times$ rather than $4\times$, because the highest-resolution mip is a small fraction of the total texel count and the lower mips remain at their original size). Second, doubling the prefilter sample count from 1024 to 2048 (rows 4 $\rightarrow$ 5) increases the prefilter pass from 33.5 to 63.2 ms ($1.9\times$, close to the expected linear scaling). Third, doubling the irradiance resolution from 32^2 to 64^2 (rows 2 $\rightarrow$ 3) takes the irradiance pass from 43.5 to 65.4 ms ($1.5\times$); the irradiance shader's inner Riemann loop is the same per texel, so the increase reflects the $4\times$ growth in texel count partly absorbed by the GPU's spare capacity at small kernel sizes. Fourth, the BRDF LUT cost is constant at ~ 8 ms regardless of any other parameter, as expected since it is material- and environment-independent.

Even at the most expensive operating point the total preprocess-

Table 1: Preprocessing time (ms) at five operating points on an Apple Silicon Mac, single HDR environment, measured with `glFinish()` barriers. Row 1 includes a one-time shader-compilation overhead on first use; rows 2–5 are warm.

#	Cubemap	Irradiance	Prefilter	Samples	Equirect	Irradiance	Prefilter	BRDF LUT	Total
1 (cold)	512 ²	32 ²	128 ²	1024	59.8	175.9	43.3	21.2	300.1
2	1024 ²	32 ²	128 ²	1024	29.4	43.5	22.0	8.1	102.9
3	1024 ²	64 ²	128 ²	1024	26.2	65.4	21.8	7.9	121.4
4	1024 ²	64 ²	256 ²	1024	57.9	63.2	33.5	7.9	162.5
5	1024 ²	64 ²	256 ²	2048	23.7	67.2	63.2	7.9	162.0

ing cost remains well under one second, which is acceptable for an interactive environment switch.

5.3. Rendering performance

The PBR fragment shader performs a constant amount of work per pixel: three texture fetches, two normalisations, one half-vector evaluation, and the Cook–Torrance arithmetic. At 1280 × 720 on the same machine the application runs at a sustained 120 FPS in both the single-sphere mode and the 7 × 7 grid mode (49 spheres, ~ 0.7 M triangles after meshing at 64 × 64 segments per sphere). This rate is the refresh-locked ceiling of the ProMotion display and represents an upper bound rather than a measure of the shader’s intrinsic cost; nevertheless it confirms that the per-pixel evaluation of equation (4) is well within the real-time budget even when the rest of the geometry pipeline is moderately loaded.

5.4. Trade-offs

The split-sum factorisation introduces a small bias that is most visible at grazing angles on rough surfaces; the assumption $\mathbf{n} = \mathbf{v} = \mathbf{r}$ during the prefilter pass loses the elongation of the GGX specular lobe in oblique reflections. In practice this error is masked by the high entropy of natural HDR content and is generally not perceptible against the more dominant effects of tonemapping and exposure.

The free parameters of the pipeline interact as follows:

- The *cubemap resolution* controls the angular fidelity of the source environment and bounds the maximum quality reachable by the prefilter pass.
- The *irradiance resolution* can be very low (16–64) without any perceptible loss, because the cosine kernel is strongly low-pass.
- The *prefilter base resolution* and *mip count* control the angular precision of mirror-like reflections; the lowest-roughness mip contains the highest-frequency content and benefits most from increased resolution.
- The *importance sample count* controls residual high-frequency noise at intermediate-to-high roughness; for $\alpha \geq 0.5$ the angular density of useful samples drops sharply, and 2048 samples remain a useful upper bound on production engines.

Of these, only the prefilter cost grows linearly with the sample count and quadratically with the base resolution; the others are negligible.

6. Limitations and Future Work

The implementation evaluates a single bounce of environmental light. It does not model parallax inside the environment (the HDR map is assumed to be at infinity), local shadowing of the IBL contribution (which would require an ambient occlusion or screen-space visibility pass), or interreflection between scene objects. The diffuse irradiance is integrated by a uniform (ϕ, θ) Riemann sum rather than projected onto spherical harmonics [RH01]; the latter would compress the storage from a cubemap to a 9-coefficient vector at no perceptual cost for the diffuse term.

The Karis prefilter assumption $\mathbf{n} = \mathbf{v}$ is the principal source of approximation error in the specular term. It can be partially mitigated by the multi-scattering formulation of Fdez-Agüera [FA19], which recovers the energy lost by the single-scattering Smith geometry term and is critical for high-roughness metals.

7. Conclusion

We described a complete real-time IBL pipeline based on the split-sum approximation, validated its behaviour qualitatively across three HDR environments with different lighting characteristics, and quantified its preprocessing and rendering cost. On an Apple Silicon Mac the technique attains the vsync-locked frame rate of 120 FPS at 1280 × 720 with a one-time preprocessing budget of approximately 100–160 ms per environment switch in warm state, and exposes a small set of intuitive parameters that allow the user to trade preprocessing time against angular fidelity. The source code, build instructions, and the data used to produce this report are available under the MIT licence.

References

- [Bur12] BURLEY B.: Physically-based shading at Disney. In *ACM SIGGRAPH 2012 Courses: Physically Based Shading in Theory and Practice* (2012). 1
- [CT82] COOK R. L., TORRANCE K. E.: A reflectance model for computer graphics. *ACM Transactions on Graphics* 1, 1 (1982), 7–24. 1
- [Deb98] DEBEVEC P. E.: Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography. In *Proceedings of SIGGRAPH '98* (1998), pp. 189–198. 1
- [Deb06] DEBEVEC P. E.: Image-based lighting. In *ACM SIGGRAPH 2006 Courses* (2006). 1
- [FA19] FDEZ-AGÜERA C. J.: A multiple-scattering microfacet model for real-time image-based lighting. *Journal of Computer Graphics Techniques* 8, 1 (2019), 45–55. 4

- [Hab10] HABLE J.: Uncharted 2: HDR lighting. In *Game Developers Conference* (2010). [3](#)
- [Hei14] HEITZ E.: Understanding the masking-shadowing function in microfacet-based BRDFs. *Journal of Computer Graphics Techniques* 3, 2 (2014), 48–107. [1](#)
- [HH64] HAMMERSLEY J. M., HANDSCOMB D. C.: *Monte Carlo Methods*. Methuen, London, 1964. [2](#)
- [Kar13] KARIS B.: Real shading in Unreal Engine 4. In *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice* (2013). [1](#), [2](#)
- [Laz13] LAZAROV D.: Getting more physical in Call of Duty: Black Ops II. In *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice* (2013). [3](#)
- [RH01] RAMAMOORTHY R., HANRAHAN P.: An efficient representation for irradiance environment maps. In *Proceedings of SIGGRAPH '01* (2001), pp. 497–500. [1](#), [4](#)
- [WMLT07] WALTER B., MARSCHNER S. R., LI H., TORRANCE K. E.: Microfacet models for refraction through rough surfaces. In *Proceedings of the Eurographics Symposium on Rendering* (2007), pp. 195–206. [1](#)